ICPC TEAM BOOK

Los Maquinolas — Universidad Mayor de San Andrés

Strings

KMP Prefix Function and Search

O(n + m)

strings, pattern matching, prefix function

```
#include <bits/stdc++.h>
using namespace std;

vector<int> prefix_function(const string &s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_search(const string &text, const string &
    pattern) {
    if (pattern.empty()) {
        vector<int> matches(text.size() + 1);
        iota(matches.begin(), matches.end(), 0);
        return matches;
    }
    vector<int> pi = prefix_function(pattern), matches;
    for (int i = 0, j = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j -
    1];
        if (text[i] == pattern[j]) ++j;
        if (j == (int)pattern.size()) {
            matches.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return matches;
}
```

Z-Function

O(n)

strings, prefix matching

```
#include <bits/stdc++.h>
using namespace std;

vector<int> z_function(const string &s) {
    int n = (int)s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i
    ];
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    if (n > 0) z[0] = n;
    return z;
}
```

Suffix Automaton

O(n) states and transitions (fixed alphabet)

strings, substrings, occurrence counts

```
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int link = -1, len = 0;
        long long occurrences = 0;
        array<int, 26> next;
        State() { next.fill(-1); }
    };
    vector<State> st;
    int last = 0;

    explicit SuffixAutomaton(int max_length = 0) {
        st.reserve(2 * max_length + 1);
        st.emplace_back();
    }

    void extend(char ch) {
        int c = ch - 'a';
        assert(0 <= c && c < 26); // This template uses the
     lowercase English alphabet.
        int cur = (int)st.size(); st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].occurrences = 1;
        int p = last;
        while (p != -1 && st[p].next[c] == -1) st[p].next[c]
     = cur, p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q
    ;
            else {
                int clone = (int)st.size(); st.push_back(st[
    q]);
                st[clone].len = st[p].len + 1;
                st[clone].occurrences = 0;
                while (p != -1 && st[p].next[c] == q) st[p].
    next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }

    void build(const string &s) { for (char c : s) extend(c)
    ; }

    void propagate_occurrences() {
        vector<int> order(st.size()); iota(order.begin(),
    order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b) {
     return st[a].len > st[b].len; });
        for (int v : order) if (st[v].link != -1)
            st[st[v].link].occurrences += st[v].occurrences;
    }

    bool contains(const string &pattern) const {
        int v = 0;
        for (char ch : pattern) {
            int c = ch - 'a';
            if (c < 0 || c >= 26 || st[v].next[c] == -1)
        return false;
            v = st[v].next[c];
        }
        return true;
    }
};
```

Aho-Corasick

O(N+M)

string matching, trie, aho-corasick

```
/*
@title: Aho-Corasick
@category: Strings
@complexity: O(N+M)
@tags: string matching, trie, aho-corasick
*/

#include <bits/stdc++.h>
using namespace std;

/*
Para frecuencias
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
  int nodes;
  vector<int> suf;
  vector<int> freq;
  vector<int> by_level;
  vector<bool> terminal;
  vector<array<int, E>> go;

  AhoCorasick() {
    add_node();
    nodes = 1;
  }

  void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    freq.emplace_back();
  }

  void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (go[pos][c] == 0) {
        go[pos][c] = nodes++;
        add_node();
      }
      pos = go[pos][c];
    }
    terminal[pos] = true;
  }

  void build() {
```

```
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
      int u = Q.front(); Q.pop();
      by_level.emplace_back(u);
      for (int c = 0; c < E; ++c) {
        if (go[u][c]) {
          int v = go[u][c];
          Q.emplace(v);
          suf[v] = u == 0 ? 0 : go[suf[u]][c];
        }
        else {
          go[u][c] = u == 0 ? 0 : go[suf[u]][c];
        }
      }
    }
  }
};*/

// Para existencia
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
  int nodes;
  vector<int> suf;
  vector<int> super;
  vector<bool> terminal;
  vector<array<int, E>> go;

  AhoCorasick() {
    add_node();
    nodes = 1;
  }

  void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    super.emplace_back();
  }

  void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (go[pos][c] == 0) {
        go[pos][c] = nodes++;
        add_node();
      }
      pos = go[pos][c];
    }
    terminal[pos] = true;
  }

  void build() {
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
      int u = Q.front(); Q.pop();
      super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] :
      super[suf[u]]);
      for (int c = 0; c < E; ++c) {
        if (go[u][c]) {
          int v = go[u][c];
          Q.emplace(v);
          suf[v] = u == 0 ? 0 : go[suf[u]][c];
        }
        else {
          go[u][c] = u == 0 ? 0 : go[suf[u]][c];
        }
      }
    }
  }
};

template<int E = 26>
void mark(int u, AhoCorasick<E> &AC) {
  if (u == 0) return;
  mark(AC.super[u], AC);
  if (AC.terminal[u]) {
    cout << "El nodo " << u << " ocurre como subcadena" <<
     endl;
    AC.terminal[u] = false;
  }
  AC.super[u] = 0;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  AhoCorasick<26> Solver;
  vector<string> S = {"arco", "co", "barco", "oro", "toro"};
  for (string s : S) {
    Solver.insert(s);
  }
  Solver.build();
  const string T = "coroparcotoro";
  int p = 0;
  for (char ch : T) {
    int c = ch - 'a';
    p = Solver.go[p][c];
    mark(p, Solver);
  }
  /*
  for (int i = (int)Solver.by_level.size() - 1; i > 0; --i) {
    int x = Solver.by_level[i];
    Solver.freq[Solver.suf[x]] += Solver.freq[x];
  }
  for (int i = 0; i < Solver.nodes; ++i) {
    cout << "El nodo " << i << " ocurre " << Solver.freq[i]
     << " veces" << endl;
  }*/
  return 0;
}
```

Manacher's Algorithm

O(N)

palindrome, string manipulation

```
/*
@title: Manacher's Algorithm
@category: Strings
@complexity: O(N)
@tags: palindrome, string manipulation
*/

vector<int> manacher_odd(string s) {
    int n = s.size();
    vector<int> radius(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r ? 1 : min(radius[l + r - i], r - i +
     1));
        while (i - k >= 0 && i + k < n && s[i - k] == s[i +
     k]) ++k;
        radius[i] = k;
        if (i + k - 1 > r) l = i - k + 1, r = i + k - 1;
    }
    return radius;
}

vector<int> manacher(string s) {
    if (s.empty()) return {};
    // Integer separators cannot collide with any byte in the
      input string.
    vector<int> t(2 * s.size() + 1, 0);
    for (int i = 0; i < (int)s.size(); ++i)
        t[2 * i + 1] = (unsigned char)s[i] + 1;
    int n = (int)t.size();
    vector<int> radius(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r ? 1 : min(radius[l + r - i], r - i +
     1));
        while (i - k >= 0 && i + k < n && t[i - k] == t[i +
     k]) ++k;
        radius[i] = k;
        if (i + k - 1 > r) l = i - k + 1, r = i + k - 1;
    }
    return vector<int>(radius.begin() + 1, radius.end() - 1)
     ;
}
```

Suffix Array

O(N log N)

suffix array, string matching

```
/*
@title: Suffix Array
@category: Strings
@complexity: O(N log N)
@tags: suffix array, string matching
*/

#include <bits/stdc++.h>
using namespace std;

vector<int> suffix_array(string s) {
  s += char(0);
  const int n = s.size();
  vector<int> a(n);
  iota(a.begin(), a.end(), 0);
  sort(a.begin(), a.end(), [&] (int i, int j) {
    return s[i] < s[j];
  });
  vector<int> mapping(n);
  mapping[a[0]] = 0;
  for(int i = 1; i < n; i++) mapping[a[i]] = mapping[a[i -
     1]] + (s[a[i - 1]] != s[a[i]]);
  int len = 1;
  vector<int> sbs(n);
  vector<int> head(n);
  vector<int> new_mapping(n);
  while(len < n) {
    for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n
     ;
    for(int i = n - 1; i >= 0; i--) head[mapping[a[i]]] = i;
    for(int i = 0; i < n; i++) {
      int x = sbs[i];
      a[head[mapping[x]]++] = x;
```

```
    }
    new_mapping[a[0]] = 0;
    for(int i = 1; i < n; i++) {
      if(mapping[a[i - 1]] != mapping[a[i]]) {
        new_mapping[a[i]] = new_mapping[a[i - 1]] + 1;
      }
      else {
        int pre = mapping[(a[i - 1] + len) % n];
        int cur = mapping[(a[i] + len) % n];
        new_mapping[a[i]] = new_mapping[a[i - 1]] + (pre !=
     cur);
      }
    }
    len <<= 1;
    swap(mapping, new_mapping);
  }
  return vector<int>(a.begin() + 1, a.end()); // Ignores a
     [0] since it's sentinel character.
}

vector<int> lcp_array(vector<int> &a, string s) {
  const int n = s.size();
  if (n == 0) return {};
  vector<int> rank(n);
  for(int i = 0; i < n; i++) rank[a[i]] = i;
  int k = 0;
  vector<int> lcp(n);
  for(int i = 0; i < n; i++) {
    if(rank[i] == n - 1) {
      k = 0;
      continue;
    }
    int j = a[rank[i] + 1];
    while(i + k < n and j + k < n and s[i + k] == s[j + k])
     k++;
    lcp[rank[i]] = k;
    if(k) k--;
  }
  lcp.pop_back();
  return lcp;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  vector<int> a = suffix_array("abcbcba");
  vector<int> lcp = lcp_array(a, "abcbcba");
  for(auto x : a) cout << x << " \n"[x == a.back()];
  for(auto x : lcp) cout << x << " \n"[x == lcp.back()];
  return 0;
}
```

Math

Linear Diophantine Equation

O(log min(|a|,|b|))

extended gcd, ax+by=c, integer solutions

```
/*
    Solve:
        a*x + b*y = c
    All solutions:
        x = x0 + k*(b/g)
        y = y0 - k*(a/g)
    Returns false if no solution exists.
    Assumes a and b are not both zero.
*/
using ll = long long;

ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

ll mygcd(ll a, ll b) {
    while (b) { a %= b; swap(a, b); }
    return a;
}

// Returns g = gcd(a,b), any solution x0,y0 to a*x0+b*y0=c.
bool find_any_solution(ll a, ll b, ll c, ll& x0, ll& y0, ll&
     g) {
    if (a == 0 && b == 0)
        return false;
    g = mygcd(llabs(a), llabs(b));
    if (c % g != 0)
        return false;
    ll x, y;
    extgcd(llabs(a), llabs(b), x, y);
    x0 = x * (c / g);
    y0 = y * (c / g);
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

ll floor_div(ll a, ll b) {
    ll q = a / b, r = a % b;
    if (r != 0 && ((r > 0) != (b > 0))) --q;
    return q;
}

ll ceil_div(ll a, ll b) { return -floor_div(-a, b); }

// Finds the range of x-values (lx..rx) among all solutions
     of a*x+b*y=c
// with x in [minx,maxx] and y in [miny,maxy].
// Number of such solutions = (rx-lx)/abs(b/g) + 1.
// NOTE: correctly handles negative/zero a or b (unlike the
     naive
// cp-algorithms-style version, which silently breaks on
     negative coeffs).
bool find_solution_range(
    ll a, ll b, ll c,
    ll minx, ll maxx,
    ll miny, ll maxy,
    ll& lx, ll& rx
) {
    ll x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return false;
    a /= g;
    b /= g;

    ll klo = LLONG_MIN / 4;
    ll khi = LLONG_MAX / 4;

    // minx <= x + k*b <= maxx
    if (b > 0) {
        klo = max(klo, ceil_div(minx - x, b));
        khi = min(khi, floor_div(maxx - x, b));
    } else if (b < 0) {
        klo = max(klo, ceil_div(maxx - x, b));
        khi = min(khi, floor_div(minx - x, b));
    } else if (x < minx || x > maxx) {
        return false; // b == 0 (a != 0): x is fixed, must
     lie in [minx,maxx]
    }

    // miny <= y - k*a <= maxy
    if (a > 0) {
        klo = max(klo, ceil_div(y - maxy, a));
        khi = min(khi, floor_div(y - miny, a));
    } else if (a < 0) {
        klo = max(klo, ceil_div(y - miny, a));
        khi = min(khi, floor_div(y - maxy, a));
    } else if (y < miny || y > maxy) {
        return false; // a == 0 (b != 0): y is fixed, must
     lie in [miny,maxy]
    }

    if (klo > khi)
        return false;

    lx = x + klo * b;
    rx = x + khi * b;
    if (lx > rx) swap(lx, rx);
    return true;
}
```

Miller-Rabin Primality Test

O(log^3 n) worst-case, 64-bit

primality, modular exponentiation

```
using ull = unsigned long long;
using u128 = __uint128_t;

ull mod_pow(ull a, ull e, ull mod) {
    ull r = 1;
    while (e) {
        if (e & 1)
            r = (u128)r * a % mod;
        a = (u128)a * a % mod;
        e >>= 1;
    }
    return r;
}

bool isPrime(ull n) {
    if (n < 2)
        return false;
for (ull p : {
        2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
        17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL
    }) {
        if (n % p == 0)
            return n == p;
    }
```

```
    ull d = n - 1;
    int s = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }
    // Deterministic witness set for all unsigned 64-bit
     integers.
    for (ull a : {
        2ULL, 325ULL, 9375ULL, 28178ULL,
        450775ULL, 9780504ULL, 1795265022ULL
    }) {
        a %= n;
        if (a == 0)
            continue;
        ull x = mod_pow(a, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool witness = true;
        for (int r = 1; r < s; ++r) {
            x = (u128)x * x % n;
            if (x == n - 1) {
                witness = false;
                break;
            }
        }
        if (witness)
            return false;
    }
    return true;
}
```

XOR Basis (64-bit)

O(64) per insertion, representation, or query

linear basis, rank, representation, maximum XOR, subset counting

```
#include <bits/stdc++.h>
using namespace std;

struct XorBasis {
    using ull = unsigned long long;
    static constexpr int BITS = 64;

    // basis[b] has highest set bit b. coefficient[b]
     expresses that vector
    // as a subset of the independent vectors in insertion
     order.
    array<ull, BITS> basis{};
    array<ull, BITS> coefficient{};
    int rank = 0;

    // Returns true exactly when x increases the rank.
    bool insert(ull x) {
        ull used = (rank < BITS ? (1ULL << rank) : 0);
        for (int bit = BITS - 1; bit >= 0; --bit) {
            if (((x >> bit) & 1ULL) == 0) continue;
            if (basis[bit]) {
                x ^= basis[bit];
                used ^= coefficient[bit];
            } else {
                basis[bit] = x;
                coefficient[bit] = used;
                ++rank;
                return true;
            }
        }
        return false;
    }

    // If x is representable, writes the subset mask in
     independent-vector
    // insertion order to mask. A 64-bit input has at most 64
      such vectors.
    bool representation(ull x, ull &mask) const {
        mask = 0;
        for (int bit = BITS - 1; bit >= 0; --bit) {
            if (((x >> bit) & 1ULL) == 0) continue;
            if (!basis[bit]) return false;
            x ^= basis[bit];
            mask ^= coefficient[bit];
        }
        return x == 0;
    }

    bool contains(ull x) const {
        ull unused;
        return representation(x, unused);
    }

    // Maximum or minimum value among seed XOR every subset-
     XOR in the basis.
    ull max_xor(ull seed = 0) const {
        for (int bit = BITS - 1; bit >= 0; --bit)
            seed = max(seed, seed ^ basis[bit]);
        return seed;
    }

    ull min_xor(ull seed = 0) const {
        for (int bit = BITS - 1; bit >= 0; --bit)
            seed = min(seed, seed ^ basis[bit]);
        return seed;
    }

    static ull pow_mod(ull a, ull e, ull mod) {
        assert(mod > 0);
        ull result = 1 % mod;
        while (e) {
            if (e & 1) result = (unsigned __int128)result *
     a % mod;
            a = (unsigned __int128)a * a % mod;
            e >>= 1;
        }
        return result;
    }

    // Counts subsets of the original n input values whose
     XOR equals target.
    // Requires mod > 0 and n >= rank.
    ull count_subsets_mod(int n, ull target, ull mod) const
     {
        assert(mod > 0 && n >= rank);
        if (!contains(target)) return 0;
        return pow_mod(2, n - rank, mod);
    }
};
```

FFT / NTT Polynomial Multiplication

O(n log n)

convolution, polynomial multiplication, FFT, NTT

```
using ll = long long;
using cd = complex<double>;
const double PI = acos(-1.0);

/* ======================== FFT ======================== */

void fft(vector<cd>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang =
            2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (cd& x : a)
            x /= n;
    }
}

vector<ll> multiply_fft(const vector<ll>& a, const vector<ll
    >& b) {
    if (a.empty() || b.empty())
        return {};
    int need = a.size() + b.size() - 1;
    int n = 1;
    while (n < need)
        n <<= 1;
    vector<cd> fa(a.begin(), a.end());
    vector<cd> fb(b.begin(), b.end());
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    vector<ll> res(need);
    for (int i = 0; i < need; i++)
        res[i] = llround(fa[i].real());

    return res;
}

/* ======================== NTT ======================== */

const ll MOD = 998244353;
const ll ROOT = 3;
```

```
ll mod_pow(ll a, ll e) {
    ll r = 1;
    while (e) {
        if (e & 1)
            r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

void ntt(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll wlen = mod_pow(
            ROOT,
            (MOD - 1) / len
        );
        if (invert)
            wlen = mod_pow(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j];
                ll v = a[i + j + len / 2] * w % MOD;
                a[i + j] = u + v < MOD
                    ? u + v
                    : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0
                    ? u - v
                    : u - v + MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll inv_n = mod_pow(n, MOD - 2);
        for (ll& x : a)
            x = x * inv_n % MOD;
    }
}

vector<ll> multiply_ntt(const vector<ll>& a, const vector<ll
    >& b) {
    if (a.empty() || b.empty())
        return {};
    int need = a.size() + b.size() - 1;
    int n = 1;
    while (n < need)
        n <<= 1;
    vector<ll> fa(a.begin(), a.end());
    vector<ll> fb(b.begin(), b.end());
    fa.resize(n);
    fb.resize(n);
    ntt(fa, false);
    ntt(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = fa[i] * fb[i] % MOD;
    ntt(fa, true);
    fa.resize(need);
    return fa;
}
```

Linear Sieve with Möbius Function

O(n)

number theory, sieve, Möbius

```
#include <bits/stdc++.h>
using namespace std;

struct MobiusSieve {
    vector<int> mu, primes;
    vector<char> composite;

    explicit MobiusSieve(int n) : mu(n + 1), composite(n +
    1, false) {
        if (n >= 1) mu[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!composite[i]) primes.push_back(i), mu[i] =
    -1;
            for (int p : primes) {
                if (1LL * i * p > n) break;
                composite[i * p] = true;
                if (i % p == 0) { mu[i * p] = 0; break; }
                mu[i * p] = -mu[i];
            }
        }
    }
};
```

Miller-Rabin and Pollard Rho Factorization

Expected

sublinear per factor for 64-bit integers

number theory, primality, factorization

```
#include <bits/stdc++.h>
using namespace std;
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mod_mul(u64 a, u64 b, u64 mod) { return (u128)a * b %
    mod; }

u64 mod_pow(u64 a, u64 e, u64 mod) {
    u64 result = 1;
    while (e) {
        if (e & 1) result = mod_mul(result, a, mod);
        a = mod_mul(a, a, mod);
        e >>= 1;
    }
    return result;
}

bool is_prime_u64(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17
    ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
    u64 d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, ++s;
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL
    , 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = mod_pow(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (u64 r = 1; r < s; ++r) {
            x = mod_mul(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}

u64 pollard_rho(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());
    while (true) {
        u64 c = rng() % (n - 1) + 1;
        u64 x = rng() % (n - 2) + 2, y = x, divisor = 1;
        auto f = [&](u64 v) { return (u64)(((u128)mod_mul(v,
     v, n) + c) % n); };
        for (int iter = 0; divisor == 1 && iter < 200000; ++
    iter) {
            x = f(x); y = f(f(y));
            u64 diff = x > y ? x - y : y - x;
            divisor = gcd(diff, n);
        }
        if (divisor > 1 && divisor < n) return divisor;
    }
}

void factorize_u64(u64 n, map<u64, int> &factors) {
    if (n == 1) return;
    if (is_prime_u64(n)) { ++factors[n]; return; }
    u64 d = pollard_rho(n);
    factorize_u64(d, factors);
    factorize_u64(n / d, factors);
}
```

Chinese Remainder Theorem

O(N)

modular arithmetic, number theory, pairwise coprime moduli

```
/*
@title: Chinese Remainder Theorem
@category: Math
@complexity: O(N)
@tags: modular arithmetic, number theory, pairwise coprime
    moduli
*/

struct Congruence {
    long long a, m;
};

// This version assumes positive, pairwise-coprime moduli and
     that their
// product fits in signed 64-bit arithmetic.
long long crt_extgcd(long long a, long long b, long long &x,
     long long &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long g = crt_extgcd(b, a % b, x1, y1);
```

```
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

long long mod_inv(long long a, long long m) {
    long long x, y;
    crt_extgcd(a, m, x, y);
    return ((x % m) + m) % m;
}

long long chinese_remainder_theorem(vector<Congruence> const
    & congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        a_i = (a_i % congruence.m + congruence.m) %
    congruence.m;
        solution = (solution + (__int128)a_i * M_i % M * N_i
    ) % M;
    }
    return solution;
}
```

Gaussian Elimination

O(N^3)

linear algebra, system of equations

```
/*
@title: Gaussian Elimination
@category: Math
@complexity: O(N^3)
@tags: linear algebra, system of equations
*/

const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity
     or a big number

int gauss (vector < vector<double> > a, vector<double> & ans
    ) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

Trees

Centroid Decomposition (Recursive)

O(n log n)

trees, centroid decomposition, distance queries

```
#include <bits/stdc++.h>
using namespace std;

struct CentroidDecompositionRecursive {
    int n;
    vector<vector<int>> adj;
    vector<int> subtree_size, centroid_parent,
     centroid_depth;
    vector<char> removed;
    vector<vector<pair<int, int>>> ancestors; // (centroid,
     distance)

    explicit CentroidDecompositionRecursive(int size)
        : n(size), adj(size), subtree_size(size),
     centroid_parent(size, -1),
          centroid_depth(size), removed(size, false),
     ancestors(size) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int calc_size(int v, int p) {
        subtree_size[v] = 1;
        for (int u : adj[v]) {
            if (u == p || removed[u]) continue;
            subtree_size[v] += calc_size(u, v);
        }
        return subtree_size[v];
    }

    int find_centroid(int v, int p, int total) {
        for (int u : adj[v]) {
            if (u == p || removed[u]) continue;
            if (subtree_size[u] > total / 2)
                return find_centroid(u, v, total);
        }
        return v;
    }

    void collect_ancestors(int v, int p, int distance, int
     centroid) {
        ancestors[v].push_back({centroid, distance});
        for (int u : adj[v]) {
            if (u == p || removed[u]) continue;
            collect_ancestors(u, v, distance + 1, centroid);
        }
    }

    void build_component(int entry, int parent_centroid, int
      depth) {
        int total = calc_size(entry, -1);
        int centroid = find_centroid(entry, -1, total);
        centroid_parent[centroid] = parent_centroid;
        centroid_depth[centroid] = depth;
        collect_ancestors(centroid, -1, 0, centroid);
        removed[centroid] = true;

        for (int u : adj[centroid]) {
            if (!removed[u])
                build_component(u, centroid, depth + 1);
        }
    }

    void build() {
        if (n > 0) build_component(0, -1, 0);
    }
};
```

Centroid Decomposition (Iterative)

O(n log n)

trees, centroid decomposition, distance queries

```
#include <bits/stdc++.h>
using namespace std;

struct CentroidDecompositionIterative {
    int n;
    vector<vector<int>> adj;
    vector<int> subtree_size, centroid_parent,
     centroid_depth;
    vector<char> removed;
    vector<vector<pair<int, int>>> ancestors; // (centroid,
     distance)

    explicit CentroidDecompositionIterative(int size)
        : n(size), adj(size), subtree_size(size),
     centroid_parent(size, -1),
          centroid_depth(size), removed(size, false),
     ancestors(size) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int find_centroid(int entry) {
        vector<int> order, parent(n, -2);
        order.push_back(entry);
        parent[entry] = -1;
        for (int i = 0; i < (int)order.size(); ++i) {
```

```
        int v = order[i];
        for (int u : adj[v]) {
            if (removed[u] || u == parent[v]) continue;
            parent[u] = v;
            order.push_back(u);
        }
    }

    for (int i = (int)order.size() - 1; i >= 0; --i) {
        int v = order[i];
        subtree_size[v] = 1;
        for (int u : adj[v])
            if (!removed[u] && parent[u] == v)
                subtree_size[v] += subtree_size[u];
    }

    int total = (int)order.size();
    for (int v : order) {
        int largest_part = total - subtree_size[v];
        for (int u : adj[v])
            if (!removed[u] && parent[u] == v)
                largest_part = max(largest_part,
 subtree_size[u]);
        if (largest_part <= total / 2) return v;
    }
    return entry;
}

void collect_ancestors(int centroid) {
    vector<tuple<int, int, int>> st = {{centroid, -1,
 0}};
    while (!st.empty()) {
        auto [v, p, d] = st.back();
        st.pop_back();
        ancestors[v].push_back({centroid, d});
        for (int u : adj[v])
            if (!removed[u] && u != p)
                st.push_back({u, v, d + 1});
    }
}

void build() {
    if (n == 0) return;
    vector<tuple<int, int, int>> tasks = {{0, -1, 0}};
    while (!tasks.empty()) {
        auto [entry, parent_centroid, depth] = tasks.
 back();
        tasks.pop_back();
        if (removed[entry]) continue;

        int centroid = find_centroid(entry);
        centroid_parent[centroid] = parent_centroid;
        centroid_depth[centroid] = depth;
        collect_ancestors(centroid);
        removed[centroid] = true;

        for (int u : adj[centroid])
            if (!removed[u])
                tasks.push_back({u, centroid, depth +
 1});
    }
}
};
```

Virtual Tree

O(k log k) including sorting

trees, LCA, tree compression

```
#include <bits/stdc++.h>
using namespace std;

// tin must be a DFS entry time, and lca(u, v) must return
     the tree LCA.
// Returns a tree over original vertex ids; callers can store
     weights separately.
vector<vector<int>> build_virtual_tree(
    vector<int> nodes,
    const vector<int> &tin,
    const function<int(int, int)> &lca
) {
    if (nodes.empty()) return vector<vector<int>>(tin.size()
    );
    sort(nodes.begin(), nodes.end(), [&](int a, int b) {
     return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.
     end());
    int k = (int)nodes.size();
    for (int i = 1; i < k; ++i) nodes.push_back(lca(nodes[i
     - 1], nodes[i]));
    sort(nodes.begin(), nodes.end(), [&](int a, int b) {
     return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.
     end());

    vector<vector<int>> tree(tin.size());
    vector<int> st;
    for (int v : nodes) {
        while (!st.empty() && lca(st.back(), v) != st.back()
     ) st.pop_back();
        if (!st.empty()) {
            tree[st.back()].push_back(v);
            tree[v].push_back(st.back());
        }
        st.push_back(v);
    }
    return tree;
}
```

Rerooting DP (Sum of Distances)

O(n)

trees, rerooting, DP

```
#include <bits/stdc++.h>
using namespace std;

// Computes, for every vertex, the sum of its distances to
     all vertices.
vector<long long> sum_distances_from_every_vertex(const
     vector<vector<int>> &tree) {
    int n = (int)tree.size();
    vector<int> parent(n, -1), order;
    vector<long long> subtree_size(n, 1), answer(n);
    if (n == 0) return answer;

    order.push_back(0);
    for (int i = 0; i < (int)order.size(); ++i) {
        int v = order[i];
        for (int u : tree[v]) if (u != parent[v]) {
            parent[u] = v;
            order.push_back(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = order[i];
        subtree_size[parent[v]] += subtree_size[v];
        answer[0] += subtree_size[v];
    }
    for (int v : order) {
        for (int u : tree[v]) if (parent[u] == v)
            answer[u] = answer[v] + n - 2 * subtree_size[u];
    }
    return answer;
}
```

DSU on Tree (Distinct Colors in Subtree)

O(n log n)

trees, sack, subtree queries

```
#include <bits/stdc++.h>
using namespace std;

// Returns the number of distinct color values in each rooted
     subtree (root 0).
vector<int> distinct_colors_in_subtrees(
    const vector<vector<int>> &tree,
    const vector<int> &color
) {
    int n = (int)tree.size();
    vector<int> parent(n, -1), size(n, 1), heavy(n, -1),
     order;
    vector<int> answer(n), compressed = color;
    sort(compressed.begin(), compressed.end());
    compressed.erase(unique(compressed.begin(), compressed.
     end()), compressed.end());
    vector<int> id(n);
    for (int v = 0; v < n; ++v)
        id[v] = lower_bound(compressed.begin(), compressed.
     end(), color[v]) - compressed.begin();
    if (n == 0) return answer;

    order.push_back(0);
    for (int i = 0; i < (int)order.size(); ++i) {
        int v = order[i];
        for (int u : tree[v]) if (u != parent[v]) {
            parent[u] = v;
            order.push_back(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = order[i], p = parent[v];
        size[p] += size[v];
        if (heavy[p] == -1 || size[v] > size[heavy[p]])
     heavy[p] = v;
    }

    vector<int> freq(compressed.size());
    int distinct = 0;
    auto add_subtree = [&](int start, int delta) {
        vector<pair<int, int>> st = {{start, parent[start
     ]}};
        while (!st.empty()) {
            auto [v, p] = st.back(); st.pop_back();
            if (delta == 1) distinct += (freq[id[v]]++ == 0)
     ;
            else distinct -= (--freq[id[v]] == 0);
            for (int u : tree[v]) if (u != p) st.push_back({
     u, v});
```

```
        }
    };
    function<void(int, bool)> solve = [&](int v, bool keep)
    {
        for (int u : tree[v]) if (parent[u] == v && u !=
    heavy[v]) solve(u, false);
        if (heavy[v] != -1) solve(heavy[v], true);
        for (int u : tree[v]) if (parent[u] == v && u !=
    heavy[v]) add_subtree(u, 1);
        distinct += (freq[id[v]]++ == 0);
        answer[v] = distinct;
        if (!keep) add_subtree(v, -1);
    };
    solve(0, true);
    return answer;
}
```

Heavy Light Decomposition

O(log^2 N)

HLD, path queries, segment tree

```
/*
@title: Heavy Light Decomposition
@category: Trees
@complexity: O(log^2 N)
@tags: HLD, path queries, segment tree
*/

#include <bits/stdc++.h>
using namespace std;

const int N = 5 * 100000 + 1;

int n;
int q;
int h[N];
int sz[N];
int par[N];
int nxt[N];
vector<int> G[N];

void DFS(int u) {
  sz[u] = 1;
  for (int i = 0; i < G[u].size(); ++i) {
    int v = G[u][i];
    DFS(v);
    sz[u] += sz[v];
    if (sz[v] > sz[G[u][0]]) {
      swap(G[u][i], G[u][0]);
    }
  }
}

void HLD(int u) {
  for (int v : G[u]) {
    h[v] = h[u] + 1;
    par[v] = u;
    nxt[v] = (v == G[u][0] ? nxt[u] : v);
    HLD(v);
  }
}

int lca(int u, int v) {
  while (nxt[u] != nxt[v]) {
    if (h[nxt[u]] > h[nxt[v]]) {
      u = par[nxt[u]];
    }
    else {
      v = par[nxt[v]];
    }
  }
  return h[u] < h[v] ? u : v;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  cin >> n >> q;
  for (int i = 1; i < n; ++i) {
    int p;
    cin >> p;
    G[p].emplace_back(i);
  }
  DFS(0);
  nxt[0] = 0;
  HLD(0);
  while (q--) {
    int u, v;
    cin >> u >> v;
    cout << lca(u, v) << '\n';
  }
  return 0;
}
```

Link-Cut Tree

O(log N) amortized per operation

dynamic trees, path queries, splay tree

```
/*
@title: Link-Cut Tree
@category: Trees
@complexity: O(log N) amortized per operation
@tags: dynamic trees, path queries, splay tree
*/

struct LinkCutTree {
    struct Node {
        int ch[2] = {0, 0}, par = 0;
        bool flip = false;
        long long val = 0, sum = 0;
    };
    vector<Node> t;

    LinkCutTree(int n) : t(n + 1) {}

    bool isRoot(int x) {
        int p = t[x].par;
        return p == 0 || (t[p].ch[0] != x && t[p].ch[1] != x
    );
    }
    void pushUp(int x) {
        t[x].sum = t[x].val + t[t[x].ch[0]].sum + t[t[x].ch
    [1]].sum;
    }
    void applyFlip(int x) {
        if (!x) return;
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].flip ^= 1;
    }
    void pushDown(int x) {
        if (t[x].flip) {
            applyFlip(t[x].ch[0]);
            applyFlip(t[x].ch[1]);
            t[x].flip = false;
        }
    }
    void rotate(int x) {
        int p = t[x].par, g = t[p].par;
        int dir = (t[p].ch[1] == x);
        if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;
        t[x].par = g;
        t[p].ch[dir] = t[x].ch[dir ^ 1];
        if (t[x].ch[dir ^ 1]) t[t[x].ch[dir ^ 1]].par = p;
        t[x].ch[dir ^ 1] = p;
        t[p].par = x;
        pushUp(p);
        pushUp(x);
    }
    void splay(int x) {
        static vector<int> stk;
        stk.clear();
        int y = x;
        stk.push_back(y);
        while (!isRoot(y)) {
            y = t[y].par;
            stk.push_back(y);
        }
        while (!stk.empty()) {
            pushDown(stk.back());
            stk.pop_back();
        }
        while (!isRoot(x)) {
            int p = t[x].par, g = t[p].par;
            if (!isRoot(p)) {
                if ((t[g].ch[1] == p) == (t[p].ch[1] == x))
     rotate(p);
                else rotate(x);
            }
            rotate(x);
        }
        pushUp(x);
    }
    // Makes x the deepest node on the root-to-x path,
     splayed to the top;
    // returns the last node touched (useful for LCA on the
     represented tree).
    int access(int x) {
        int last = 0;
        for (int y = x; y; y = t[y].par) {
            splay(y);
            t[y].ch[1] = last;
            pushUp(y);
            last = y;
        }
        splay(x);
        return last;
    }
    void makeRoot(int x) {
        access(x);
        applyFlip(x);
    }
    int findRoot(int x) {
        access(x);
        while (t[x].ch[0]) {
            pushDown(x);
            x = t[x].ch[0];
        }
```

```
        splay(x);
        return x;
    }
    bool connected(int x, int y) {
        if (x == y) return true;
        return findRoot(x) == findRoot(y);
    }
    // Links x to y. Precondition: x and y are in different
     trees.
    void link(int x, int y) {
        makeRoot(x);
        t[x].par = y;
    }
    // Cuts the edge between x and y, if it exists (safe no-
    op otherwise).
    void cut(int x, int y) {
        makeRoot(x);
        access(y);
        if (t[y].ch[0] == x && t[x].ch[1] == 0) {
            t[y].ch[0] = 0;
            t[x].par = 0;
            pushUp(y);
        }
    }
    void setVal(int x, long long v) {
        access(x);
        t[x].val = v;
        pushUp(x);
    }
    // Sum of vertex values on the path from x to y.
     Precondition: connected(x,y).
    long long pathQuery(int x, int y) {
        makeRoot(x);
        access(y);
        return t[y].sum;
    }
};
```

Graphs

2-SAT

O(n + m)

SCC, implication graph, satisfiability

```
struct TwoSat {
    int n;

    vector<vector<int>> g, rg;
    vector<int> comp, order;
    vector<char> used;
    vector<bool> assignment;

    TwoSat(int n)
        : n(n),
          g(2 * n),
          rg(2 * n),
          comp(2 * n, -1),
          order(),
          used(2 * n, false),
          assignment(n, false) {}

    int node(int x, bool neg) {
        return 2 * x + neg;
    }

    void add_clause(
        int a, bool na,
        int b, bool nb
    ) {
        int A = node(a, na);
        int B = node(b, nb);
        g[A ^ 1].push_back(B);
        g[B ^ 1].push_back(A);
        rg[B].push_back(A ^ 1);
        rg[A].push_back(B ^ 1);
    }

    void dfs1(int v) {
        used[v] = true;
        for (int u : g[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }

    void dfs2(int v, int c) {
        comp[v] = c;
        for (int u : rg[v]) {
            if (comp[u] == -1)
                dfs2(u, c);
        }
    }

    bool solve() {
        fill(used.begin(), used.end(), false);
        order.clear();
        for (int v = 0; v < 2 * n; v++) {
            if (!used[v])
                dfs1(v);
        }
        fill(comp.begin(), comp.end(), -1);
        int components = 0;
        for (int i = 2 * n - 1; i >= 0; i--) {
            int v = order[i];
            if (comp[v] == -1)
                dfs2(v, components++);
        }
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1])
                return false;
            assignment[i] = comp[2 * i] > comp[2 * i + 1];
        }
        return true;
    }
};
```

Strongly Connected Components (Kosaraju)

O(n + m)

directed graph, SCC, condensation graph

```
#include <bits/stdc++.h>
using namespace std;

struct KosarajuSCC {
    int n, components = 0;
    vector<vector<int>> g, rg;
    vector<int> order, comp;

    explicit KosarajuSCC(int size) : n(size), g(size), rg(
     size), comp(size, -1) {}

    void add_edge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    void build() {
        vector<char> seen(n, false);
        order.clear();
        function<void(int)> dfs1 = [&](int v) {
            seen[v] = true;
            for (int u : g[v]) if (!seen[u]) dfs1(u);
            order.push_back(v);
        };
        for (int v = 0; v < n; ++v) if (!seen[v]) dfs1(v);

        fill(comp.begin(), comp.end(), -1);
        components = 0;
        function<void(int)> dfs2 = [&](int v) {
            comp[v] = components;
            for (int u : rg[v]) if (comp[u] == -1) dfs2(u);
        };
        for (int i = n - 1; i >= 0; --i) {
            int v = order[i];
            if (comp[v] == -1) {
                dfs2(v);
                ++components;
            }
        }
    }

    vector<vector<int>> condensation_graph() const {
        vector<vector<int>> dag(components);
        for (int v = 0; v < n; ++v)
            for (int u : g[v])
                if (comp[v] != comp[u]) dag[comp[v]].
     push_back(comp[u]);
        for (auto &edges : dag) {
            sort(edges.begin(), edges.end());
            edges.erase(unique(edges.begin(), edges.end()),
     edges.end());
        }
        return dag;
    }
};
```

Bridges and Articulation Points

O(n + m)

undirected graph, low-link, bridges, articulation points

```
#include <bits/stdc++.h>
using namespace std;

struct BridgesArticulationPoints {
    int n, timer = 0;
    vector<vector<pair<int, int>>> adj; // (neighbor, edge
     id)
    vector<int> tin, low;
    vector<char> is_bridge, is_articulation;

    explicit BridgesArticulationPoints(int size)
        : n(size), adj(size), tin(size, -1), low(size),
     is_articulation(size, false) {}

    void add_edge(int u, int v) {
        int id = (int)is_bridge.size();
        is_bridge.push_back(false);
```

```
        adj[u].push_back({v, id});
        adj[v].push_back({u, id});
    }

    void dfs(int v, int parent_edge = -1) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [u, id] : adj[v]) {
            if (id == parent_edge) continue;
            if (tin[u] != -1) {
                low[v] = min(low[v], tin[u]);
            } else {
                dfs(u, id);
                low[v] = min(low[v], low[u]);
                if (low[u] > tin[v]) is_bridge[id] = true;
                if (parent_edge != -1 && low[u] >= tin[v])
                    is_articulation[v] = true;
                ++children;
            }
        }
        if (parent_edge == -1 && children > 1)
     is_articulation[v] = true;
    }

    void build() {
        timer = 0;
        fill(tin.begin(), tin.end(), -1);
        fill(low.begin(), low.end(), 0);
        fill(is_bridge.begin(), is_bridge.end(), false);
        fill(is_articulation.begin(), is_articulation.end(),
      false);
        for (int v = 0; v < n; ++v)
            if (tin[v] == -1) dfs(v);
    }
};
```

Kruskal Minimum Spanning Tree

O(m log m)

undirected graph, MST, DSU

```
#include <bits/stdc++.h>
using namespace std;

struct KruskalMST {
    struct Edge { int u, v; long long w; };
    int n;
    vector<Edge> edges, mst_edges;
    long long total_weight = 0;

    explicit KruskalMST(int size) : n(size) {}

    void add_edge(int u, int v, long long w) {
        edges.push_back({u, v, w});
    }

    pair<bool, long long> build() {
        vector<int> parent(n), size(n, 1);
        iota(parent.begin(), parent.end(), 0);
        auto find = [&](int x) {
            int r = x;
            while (r != parent[r]) r = parent[r];
            while (x != r) {
                int p = parent[x];
                parent[x] = r;
                x = p;
            }
            return r;
        };

        sort(edges.begin(), edges.end(), [](const Edge &a,
     const Edge &b) {
            return a.w < b.w;
        });
        mst_edges.clear();
        total_weight = 0;
        for (const Edge &e : edges) {
            int a = find(e.u), b = find(e.v);
            if (a == b) continue;
            if (size[a] < size[b]) swap(a, b);
            parent[b] = a;
            size[a] += size[b];
            mst_edges.push_back(e);
            total_weight += e.w;
        }
        return {(int)mst_edges.size() == max(0, n - 1),
     total_weight};
    }
};
```

Hopcroft-Karp Bipartite Matching

O(E sqrt(V))

bipartite matching, maximum matching

```
#include <bits/stdc++.h>
using namespace std;

struct HopcroftKarp {
    int left_size, right_size;
    vector<vector<int>> adj;
    vector<int> match_left, match_right, dist;

    int shortest_distance = INT_MAX;

    HopcroftKarp(int left_count, int right_count)
        : left_size(left_count), right_size(right_count),
     adj(left_count),
          match_left(left_count, -1), match_right(
     right_count, -1), dist(left_count) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool bfs() {
        queue<int> q;
        for (int u = 0; u < left_size; ++u) {
            if (match_left[u] == -1) dist[u] = 0, q.push(u);
            else dist[u] = -1;
        }
        int shortest = INT_MAX;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] + 1 > shortest) continue;
            for (int v : adj[u]) {
                int next = match_right[v];
                if (next == -1) shortest = dist[u] + 1;
                else if (dist[next] == -1) dist[next] = dist
     [u] + 1, q.push(next);
            }
        }
        shortest_distance = shortest;
        return shortest != INT_MAX;
    }

    bool dfs(int u) {
        for (int v : adj[u]) {
            int next = match_right[v];
            if ((next == -1 && dist[u] + 1 ==
     shortest_distance) ||
                (next != -1 && dist[next] == dist[u] + 1 &&
     dfs(next))) {
                match_left[u] = v;
                match_right[v] = u;
                return true;
            }
        }
        dist[u] = -1;
        return false;
    }

    int maximum_matching() {
        int result = count_if(match_left.begin(), match_left
     .end(), [](int v) { return v != -1; });
        while (bfs()) {
            for (int u = 0; u < left_size; ++u)
                if (match_left[u] == -1 && dfs(u)) ++result;
        }
        return result;
    }
};
```

Bellman-Ford Algorithm

O(VE)

shortest path, negative weights

```
/*
@title: Bellman-Ford Algorithm
@category: Graphs
@complexity: O(VE)
@tags: shortest path, negative weights
*/

struct Edge {
    int a, b;
    long long cost;
};

int n;
vector<Edge> edges;
void solve() {
    vector<__int128> d(n, 0);
    vector<int> p(n, -1);
    int x = -1;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            __int128 candidate = d[e.a] + e.cost;
            if (candidate < d[e.b]) {
                d[e.b] = candidate;
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }

    if (x == -1) {
        cout << "No negative cycle found.";
    } else {
        for (int i = 0; i < n; ++i)
```

```
            x = p[x];

        vector<int> cycle;
        for (int v = x;; v = p[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());

        cout << "Negative cycle: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}
```

Eulerian Path

O(V^2 + E)

graph traversal, Hierholzer's algorithm

```
/*
@title: Eulerian Path
@category: Graphs
@complexity: O(V^2 + E)
@tags: graph traversal, Hierholzer's algorithm
@input: n, then n x n adjacency matrix (undirected multigraph
    , NO self-loops)
@output: sequence of vertices of an Eulerian path/circuit, or
     -1 if none exists
*/

int main() {
    int n;
    cin >> n;
    vector<vector<int>> g(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> g[i][j];

    // Self-loops never change whether an Eulerian path/
     circuit exists (they
    // are entered and exited at the same vertex), but this
     matrix-based
    // traversal cannot consume them correctly (it decrements
      the same cell
    // twice per step and hangs forever). Strip them before
     running.
    // If your problem needs self-loops printed, remember how
      many each
    // vertex had and splice them into the output wherever
     that vertex
    // appears in the final trail.
    for (int i = 0; i < n; ++i)
        g[i][i] = 0;

    vector<int> deg(n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1 << "\n"; // no edges at all
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1) v1 = i;
            else if (v2 == -1) v2 = i;
            else bad = true;
        }
    }

    if (v1 != -1)
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i]) break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i) {
            if ((res[i] == v1 && res[i + 1] == v2) ||
                (res[i] == v2 && res[i + 1] == v1)) {
                vector<int> res2;
                for (size_t j = i + 1; j < res.size(); ++j)
                    res2.push_back(res[j]);
                for (size_t j = 1; j <= i; ++j)
                    res2.push_back(res[j]);
                res = res2;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (g[i][j]) bad = true; // leftover edges =>
     graph was disconnected

    if (bad) {
        cout << -1 << "\n";
    } else {
        for (int x : res) cout << x << " ";
        cout << "\n";
    }
}
```

Flows

Min-Cost Max-Flow

O(F E log V) with nonnegative reduced costs

flow, shortest paths, potentials

```
#include <bits/stdc++.h>
using namespace std;

struct MinCostMaxFlow {
    struct Edge { int to, rev, cap; long long cost; };
    int n;
    vector<vector<Edge>> graph;
    explicit MinCostMaxFlow(int size) : n(size), graph(size)
      {}

    void add_edge(int u, int v, int cap, long long cost) {
        int rev_u = (int)graph[v].size();
        int rev_v = (int)graph[u].size();
        if (u == v) ++rev_u;
        Edge a{v, rev_u, cap, cost};
        Edge b{u, rev_v, 0, -cost};
        graph[u].push_back(a);
        graph[v].push_back(b);
    }

    pair<int, long long> run(int source, int sink, int
     flow_limit = INT_MAX) {
        const long long INF = LLONG_MAX / 4;
        vector<long long> potential(n, 0), dist(n);
        vector<int> parent_v(n), parent_e(n);
        // Initial shortest distances support negative
     forward costs. The input
        // residual graph must not contain a source-reachable
      negative cycle.
        vector<char> in_queue(n, false);
        queue<int> initial_queue;
        fill(dist.begin(), dist.end(), INF);
        dist[source] = 0;
        initial_queue.push(source);
        in_queue[source] = true;
        while (!initial_queue.empty()) {
            int v = initial_queue.front(); initial_queue.pop
     ();
            in_queue[v] = false;
            for (const Edge &e : graph[v]) if (e.cap > 0 &&
     dist[v] + e.cost < dist[e.to]) {
                dist[e.to] = dist[v] + e.cost;
                if (!in_queue[e.to]) in_queue[e.to] = true,
     initial_queue.push(e.to);
            }
        }
        for (int v = 0; v < n; ++v) if (dist[v] < INF)
     potential[v] = dist[v];
        int flow = 0;
        long long cost = 0;
        while (flow < flow_limit) {
            fill(dist.begin(), dist.end(), INF);
            dist[source] = 0;
            priority_queue<pair<long long, int>, vector<pair
     <long long, int>>, greater<>> pq;
            pq.push({0, source});
            while (!pq.empty()) {
                auto [d, v] = pq.top(); pq.pop();
                if (d != dist[v]) continue;
```

```
            for (int i = 0; i < (int)graph[v].size(); ++
    i) {
                const Edge &e = graph[v][i];
                if (e.cap == 0) continue;
                long long nd = d + e.cost + potential[v]
     - potential[e.to];
                if (nd < dist[e.to]) {
                    dist[e.to] = nd;
                    parent_v[e.to] = v;
                    parent_e[e.to] = i;
                    pq.push({nd, e.to});
                }
            }
        }
        if (dist[sink] == INF) break;
        for (int v = 0; v < n; ++v) if (dist[v] < INF)
    potential[v] += dist[v];
        int add = flow_limit - flow;
        for (int v = sink; v != source; v = parent_v[v])
            add = min(add, graph[parent_v[v]][parent_e[v
    ]].cap);
        for (int v = sink; v != source; v = parent_v[v])
     {
            Edge &e = graph[parent_v[v]][parent_e[v]];
            cost += (long long)add * e.cost;
            e.cap -= add;
            graph[v][e.rev].cap += add;
        }
        flow += add;
      }
      return {flow, cost};
   }
};
```

Hungarian Algorithm

O(n^2 m), O(n^3) if n = m

assignment problem, minimum-cost matching

```
/*
    Minimum-cost assignment.
    A[i][j] = cost of assigning row i to column j.
    n <= m
    Returns:
        ans[i] = column assigned to row i
    Also returns the minimum total cost.
    1-indexed matrix A[1..n][1..m].
    For maximum cost:
        negate all A[i][j].
*/

pair<long long, vector<int>> hungarian(
    const vector<vector<long long>>& A
) {
    int n = (int)A.size() - 1;
    int m = (int)A[0].size() - 1;

    const long long INF = (1LL << 60);

    vector<long long> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);

    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<long long> minv(m + 1, INF);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0];
            long long delta = INF;
            int j1 = 0;
            for (int j = 1; j <= m; j++) {
                if (used[j]) continue;

                long long cur = A[i0][j] - u[i0] - v[j];

                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }

                if (minv[j] < delta) {
                    delta = minv[j];
                    j1 = j;
                }
            }

            for (int j = 0; j <= m; j++) {
                if (used[j]) {
                    u[p[j]] += delta;
                    v[j] -= delta;
                } else {
                    minv[j] -= delta;
                }
            }
            j0 = j1;
        } while (p[j0] != 0);

        // Restore the augmenting path.
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;

        } while (j0);
    }

    // p[j] = row assigned to column j.
    // Convert to ans[i] = column assigned to row i.
    vector<int> ans(n + 1);

    for (int j = 1; j <= m; j++)
        ans[p[j]] = j;

    return {-v[0], ans};
}
```

Dinic's Algorithm

O(V^2 * E), O(E * sqrt(V)) for bipartite graphs

max flow, residual graph

```
/*
@title: Dinic's Algorithm
@category: Graphs
@complexity: O(V^2 * E), O(E * sqrt(V)) for bipartite graphs
@tags: max flow, residual graph
*/

struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(
        cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size();
     cid++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap
     - edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }
```

```
    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

Geometry

Faces of a Planar Graph

O(m log m)

planar graph, faces, embedding

```
/*
    Input:
        - Connected planar graph
        - Straight-line embedding
        - vertices[i] = coordinates of vertex i
        - adj[i] = neighbors of vertex i

    Output:
        faces[0] = outer face
        faces[i] = inner faces

    Inner faces are CCW.
    Outer face is CW.

    Vertices are 0-indexed.

    For a planar graph:
        m = O(n)
    so complexity is also O(n log n).

    If adjacency lists are already sorted by angle,
    the traversal itself can be made O(n).
*/

struct Point {
    long long x, y;

    Point(long long x = 0, long long y = 0)
        : x(x), y(y) {}

    Point operator-(const Point& p) const {
        return Point(x - p.x, y - p.y);
    }

    long long cross(const Point& p) const {
        return x * p.y - y * p.x;
    }

    int half() const {
        return y < 0 || (y == 0 && x < 0);
    }
};

vector<vector<int>> planar_faces(
    vector<Point> p,
    vector<vector<int>> adj
) {
    int n = p.size();

    // Sort neighbors counter-clockwise by polar angle.
    vector<vector<char>> used(n);

    for (int i = 0; i < n; i++) {
        used[i].assign(adj[i].size(), false);

        auto cmp = [&](int l, int r) {
            Point a = p[l] - p[i];
            Point b = p[r] - p[i];

            if (a.half() != b.half())
                return a.half() < b.half();

            return a.cross(b) > 0;
        };

        sort(adj[i].begin(), adj[i].end(), cmp);
    }

    vector<vector<int>> faces;

    // Every directed edge belongs to exactly one face.
    for (int s = 0; s < n; s++) {
        for (int e0 = 0; e0 < (int)adj[s].size(); e0++) {
            if (used[s][e0])
                continue;

            vector<int> face;

            int v = s;
            int e = e0;

            while (!used[v][e]) {
                used[v][e] = true;
                face.push_back(v);

                int u = adj[v][e];

                // Find v in adj[u].
                auto cmp = [&](int l, int r) {
                    Point a = p[l] - p[u];
                    Point b = p[r] - p[u];

                    if (a.half() != b.half())
                        return a.half() < b.half();

                    return a.cross(b) > 0;
                };

                int pos = lower_bound(
                    adj[u].begin(),
                    adj[u].end(),
                    v,
                    cmp
                ) - adj[u].begin();

                // Take the next edge clockwise.
                e = (pos + 1) % adj[u].size();
                v = u;
            }

            reverse(face.begin(), face.end());

            // Positive signed area -> inner face.
            // Non-positive -> outer face.
            Point p0 = p[face[0]];
            __int128 area = 0;

            for (int i = 0; i < (int)face.size(); i++) {
                Point a = p[face[i]];
                Point b = p[face[(i + 1) % face.size()]];

                area += (a - p0).cross(b - a);
            }

            if (area <= 0)
                faces.insert(faces.begin(), face);
            else
                faces.push_back(face);
        }
    }

    return faces;
}
```

Convex Hull (Monotone Chain)

O(n log n)

geometry, convex hull, integer points

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    long long x, y;
    bool operator<(const Point &other) const { return tie(x,
      y) < tie(other.x, other.y); }
    bool operator==(const Point &other) const { return x ==
     other.x && y == other.y; }
};

__int128 cross(const Point &a, const Point &b, const Point &
    c) {
    return ((__int128)b.x - a.x) * ((__int128)c.y - a.y)
         - ((__int128)b.y - a.y) * ((__int128)c.x - a.x);
}

// Returns the convex hull counterclockwise, excluding
    collinear points on edges.
vector<Point> convex_hull(vector<Point> points) {
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()),
     points.end());
    if (points.size() <= 1) return points;
    vector<Point> hull;
    for (const Point &p : points) {
        while (hull.size() >= 2 && cross(hull[hull.size() -
     2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    size_t lower_size = hull.size();
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        const Point &p = points[i];
        while (hull.size() > lower_size && cross(hull[hull.
```

```
    size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    hull.pop_back();
    return hull;
}
```

Dynamic Programming

Divide & Conquer DP

O(m n log n)

DP optimization, monotone opt

```
/*
    DP of the form:
        dp[g][j] = min_{g-1 <= k <= j} dp[g-1][k-1] + C(k,j)
    where opt[i][j] is monotone:
        opt[i][j] <= opt[i][j+1]
    Sufficient condition:
        C(a,c) + C(b,d) <= C(a,d) + C(b,c)
        for a <= b <= c <= d.
    IMPORTANT:
        solve() computes exactly m nonempty groups. On layer
     i, k >= i
        guarantees each group contains at least one element.
*/

using ll = long long;

int n, m;
vector<ll> dp_before, dp_cur;

ll C(int k, int j);

void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;
    int mid = (l + r) >> 1;
    pair<ll, int> best = {LLONG_MAX,-1};
    for (int k = optl;k <= min(mid, optr);k++) {
        ll val =
            (k ? dp_before[k - 1] : 0)
            + C(k, mid);
        if (val < best.first)
            best = {val, k};
    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}

ll solve() {
    if (n <= 0 || m < 1 || m > n)
        return LLONG_MAX;
    dp_before.assign(n, 0);
    dp_cur.assign(n, 0);
    for (int j = 0; j < n; j++)
        dp_before[j] = C(0, j);
    const ll INF = LLONG_MAX;
    for (int i = 1; i < m; i++) {
        fill(dp_cur.begin(), dp_cur.end(), INF);
        compute(i, n - 1, i, n - 1);
        dp_before = dp_cur;
    }
    return dp_before[n - 1];
}
```

Knuth Optimization

O(n^2)

interval DP, DP optimization

```
/*
    dp[l][r] = minimum cost for interval [l,r).
    Transition:
        dp[l][r] = min(dp[l][k] + dp[k][r] + C(l,r))   for l
     < k < r
    Knuth condition:
        opt[l][r-1] <= opt[l][r] <= opt[l+1][r]
    Base:
        dp[i][i] = 0, dp[i][i+1] = 0 (single element, no
     split needed)
    Requires C to satisfy the quadrangle inequality +
     monotonicity
    (true e.g. for C(l,r) = prefix[r]-prefix[l] with non-
     negative weights).
*/

using ll = long long;
vector<vector<ll>> knuth(int n, const vector<ll>& prefix) {
    const ll INF = (1LL << 62);
    vector<vector<ll>> dp(n + 1, vector<ll>(n + 1, 0));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1, 0));

    for (int i = 0; i <= n; i++)
        opt[i][i] = i;
    for (int i = 0; i < n; i++)
        opt[i][i + 1] = i;   // seed the length-1 boundary (
     was left at 0 before!)

    auto C = [&](int l, int r) -> ll {
        return prefix[r] - prefix[l];
    };

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len;
            dp[l][r] = INF;
            int L = max(opt[l][r - 1], l + 1);
            int R = min(opt[l + 1][r], r - 1);
            for (int k = L; k <= R; k++) {
                ll val = dp[l][k] + dp[k][r] + C(l, r);
                if (val < dp[l][r]) {
                    dp[l][r] = val;
                    opt[l][r] = k;
                }
            }
        }
    }

    return dp;
}
```

SOS DP (Subset Sums)

O(n 2^n)

bitmask DP, subset aggregation

```
#include <bits/stdc++.h>
using namespace std;

// Afterward, subset_sum[mask] is the sum of freq[sub] over
     all submasks sub of mask.
vector<long long> sos_subset_sums(vector<long long> freq,
     int bits) {
    assert(bits >= 0 && bits < 31);
    assert((int)freq.size() == (1 << bits));
    for (int bit = 0; bit < bits; ++bit)
        for (int mask = 0; mask < (1 << bits); ++mask)
            if (mask & (1 << bit)) freq[mask] += freq[mask ^
      (1 << bit)];
    return freq;
}
```

Convex Hull Trick (Monotone Maximum)

Amortized O(1) per insertion and query

DP optimization, lines, convex hull trick

```
#include <bits/stdc++.h>
using namespace std;

struct MonotoneMaxCHT {
    struct Line {
        long long m, b;
        long long eval(long long x) const { return m * x + b
     ; }
    };
    deque<Line> hull;

    // Add slopes in strictly increasing order (equal slopes
     keep the larger intercept).
    void add_line(long long m, long long b) {
        if (!hull.empty() && hull.back().m == m) {
            if (hull.back().b >= b) return;
            hull.pop_back();
        }
        while (hull.size() >= 2) {
            Line a = hull[hull.size() - 2], c = hull.back();
            if (((__int128)a.b - c.b) * ((__int128)m - c.m)
     <
                    ((__int128)c.b - b) * ((__int128)c.m - a.m))
      break;
            hull.pop_back();
        }
        hull.push_back({m, b});
    }

    // Query x values in nondecreasing order.
    long long query(long long x) {
        assert(!hull.empty());
        while (hull.size() >= 2 && hull[0].eval(x) <= hull
     [1].eval(x)) hull.pop_front();
        return hull.front().eval(x);
    }
};
```

Data Structures

DSU with Rollback

O(log n) per operation, O(1) per rollback

DSU, rollback, offline queries

```
#include <bits/stdc++.h>
using namespace std;

struct DSURollback {
    int components;
    vector<int> parent, size;
    vector<pair<int, int>> history;
```

```
    explicit DSURollback(int n) : components(n), parent(n),
     size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int v) const {
        while (v != parent[v]) v = parent[v];
        return v;
    }

    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) {
            history.push_back({-1, -1});
            return false;
        }
        if (size[a] < size[b]) swap(a, b);
        history.push_back({b, size[a]});
        parent[b] = a;
        size[a] += size[b];
        --components;
        return true;
    }

    int snapshot() const { return (int)history.size(); }

    void rollback(int snapshot) {
        while ((int)history.size() > snapshot) {
            auto [child, old_size] = history.back();
            history.pop_back();
            if (child == -1) continue;
            int root = parent[child];
            parent[child] = child;
            size[root] = old_size;
            ++components;
        }
    }
};
```

Li Chao Tree (Maximum, Discrete Coordinates) O(log X) per insertion or query

data structures, lines, DP optimization

```
#include <bits/stdc++.h>
using namespace std;

struct LiChaoTree {
    struct Line {
        long long m = 0, b = 0;
        long long eval(long long x) const { return m * x + b
     ; }
    };
    vector<long long> xs;
    vector<Line> tree;
    vector<char> has;

    explicit LiChaoTree(vector<long long> coordinates) : xs(
     move(coordinates)) {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        tree.resize(max(1, 4 * (int)xs.size()));
        has.assign(tree.size(), false);
    }

    void add_line(Line line) { if (!xs.empty()) add_line(1,
     0, (int)xs.size() - 1, line); }

    void add_line(int p, int l, int r, Line line) {
        if (!has[p]) { tree[p] = line; has[p] = true; return
     ; }
        int mid = (l + r) / 2;
        bool left = line.eval(xs[l]) > tree[p].eval(xs[l]);
        bool middle = line.eval(xs[mid]) > tree[p].eval(xs[
    mid]);
        if (middle) swap(line, tree[p]);
        if (l == r) return;
        if (left != middle) add_line(p * 2, l, mid, line);
        else add_line(p * 2 + 1, mid + 1, r, line);
    }

    long long query(long long x) const {
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.
    begin();
        assert(i < (int)xs.size() && xs[i] == x);
        return query(1, 0, (int)xs.size() - 1, i);
    }

    long long query(int p, int l, int r, int i) const {
        long long result = has[p] ? tree[p].eval(xs[i]) :
     LLONG_MIN;
        if (l == r) return result;
        int mid = (l + r) / 2;
        long long child = i <= mid ? query(p * 2, l, mid, i)
                                   : query(p * 2 + 1, mid +
     1, r, i);
        return max(result, child);
    }
};
```

Persistent Segment Tree (K-th Smallest) O(log n) per version update or range query

persistent segment tree, order statistics

```
#include <bits/stdc++.h>
using namespace std;

struct PersistentKth {
    struct Node { int left = 0, right = 0, sum = 0; };
    vector<Node> nodes = {Node()};
    vector<int> values, roots;

    explicit PersistentKth(vector<int> a) : values(move(a))
     {
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()),
     values.end());
        roots.push_back(0);
    }

    int insert(int old, int l, int r, int pos) {
        int cur = (int)nodes.size();
        nodes.push_back(nodes[old]);
        ++nodes[cur].sum;
        if (l != r) {
            int mid = (l + r) / 2;
            if (pos <= mid) nodes[cur].left = insert(nodes[
     old].left, l, mid, pos);
            else nodes[cur].right = insert(nodes[old].right,
      mid + 1, r, pos);
        }
        return cur;
    }

    void build(const vector<int> &a) {
        roots.assign(1, 0);
        for (int x : a) {
            int pos = lower_bound(values.begin(), values.end
     (), x) - values.begin();
            roots.push_back(values.empty() ? 0 : insert(
     roots.back(), 0, (int)values.size() - 1, pos));
        }
    }

    // k is 1-indexed; [l, r] is a 0-indexed inclusive range
     of the original array.
    int kth(int l, int r, int k) const {
        assert(0 <= l && l <= r && r + 1 < (int)roots.size()
     );
        assert(1 <= k && k <= nodes[roots[r + 1]].sum -
     nodes[roots[l]].sum);
        int a = roots[l], b = roots[r + 1], lo = 0, hi = (
     int)values.size() - 1;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            int count_left = nodes[nodes[b].left].sum -
     nodes[nodes[a].left].sum;
            if (k <= count_left) { a = nodes[a].left; b =
     nodes[b].left; hi = mid; }
            else { k -= count_left; a = nodes[a].right; b =
     nodes[b].right; lo = mid + 1; }
        }
        return values[lo];
    }
};
```

Lazy Segment Tree

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct LazySegTree {
    int n;
    vector<ll> st, lazy;

    LazySegTree(const vector<ll>& a) {
        n = (int)a.size();
        st.assign(4 * n, 0);
        lazy.assign(4 * n, 0);
        build(1, 0, n - 1, a);
    }

    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            st[p] = a[l];
            return;
        }

        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }
```

```
void pull(int p) {
    st[p] = st[p * 2] + st[p * 2 + 1];
}

void apply(int p, int l, int r, ll x) {
    st[p] += x * (r - l + 1);
    lazy[p] += x;
}

void push(int p, int l, int r) {
    if (lazy[p] == 0 || l == r) return;

    int m = (l + r) / 2;

    apply(p * 2, l, m, lazy[p]);
    apply(p * 2 + 1, m + 1, r, lazy[p]);

    lazy[p] = 0;
}

void update(int p, int l, int r,
            int ql, int qr, ll x) {
    if (qr < l || r < ql) return;

    if (ql <= l && r <= qr) {
        apply(p, l, r, x);
        return;
    }

    push(p, l, r);

    int m = (l + r) / 2;
    update(p * 2, l, m, ql, qr, x);
    update(p * 2 + 1, m + 1, r, ql, qr, x);

    pull(p);
}

ll query(int p, int l, int r,
         int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr)
        return st[p];

    push(p, l, r);

    int m = (l + r) / 2;

    return query(p * 2, l, m, ql, qr)
         + query(p * 2 + 1, m + 1, r, ql, qr);
}

void range_add(int l, int r, ll x) {
    if (l > r) return;
    update(1, 0, n - 1, l, r, x);
}

ll range_sum(int l, int r) {
    if (l > r) return 0;
    return query(1, 0, n - 1, l, r);
}
};
```